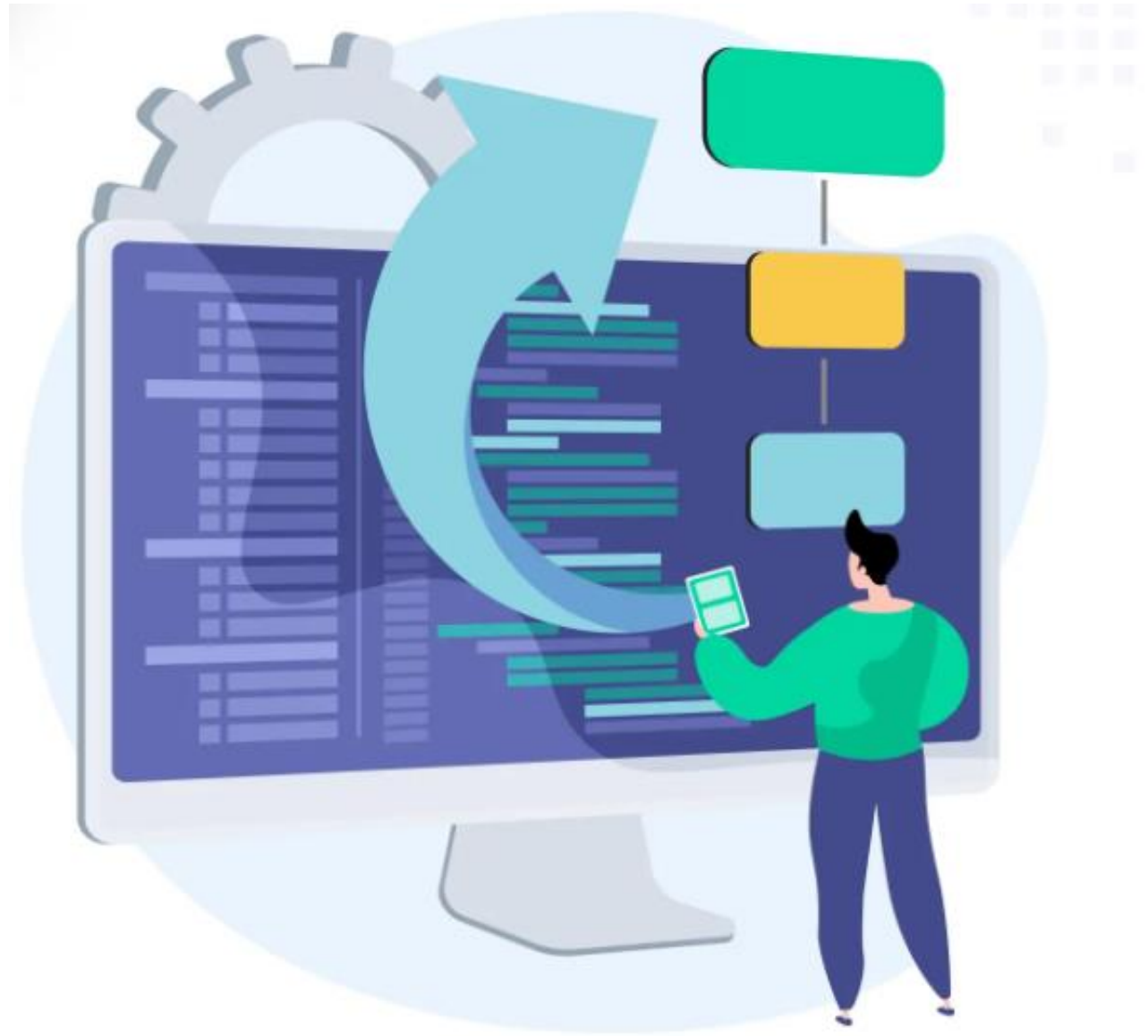


Software Requirement Specification (SRS)



Outline

- Introduction
- The goal of SRS
- Why SRS document is important
- SRS document elements
- Characteristics of a good SRS
- SRS document
- Steps to write SRS document
- What should a typical SRS document include
- Tools for SRS documents
 - Use case diagram
 - User story
 - Workflow
 - Wireframe
 - Logical data model
 - Data dictionary

Introduction

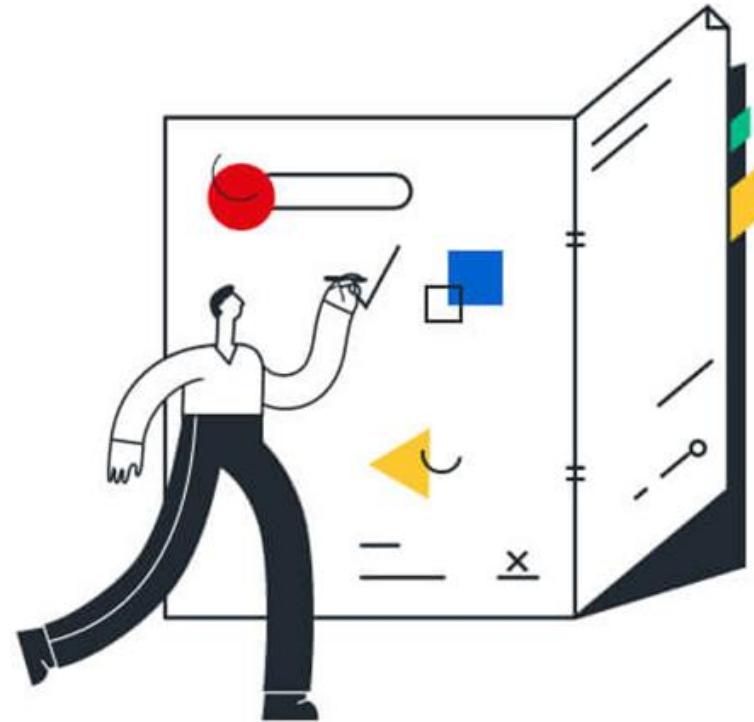
- A software requirements specification (SRS) is a **comprehensive description** of the intended purpose and environment for software under development. **The SRS fully describes what the software will do and how it will be expected to perform.**
- An SRS is a software development **roadmap** that provides designers, testers, clients and end-users with metrics, use case scenarios, and other details of the future product. **Briefly saying, it collects the clients' demands and explanations of the development team, how these demands will come to life in custom software development services.**

The goals of SRS

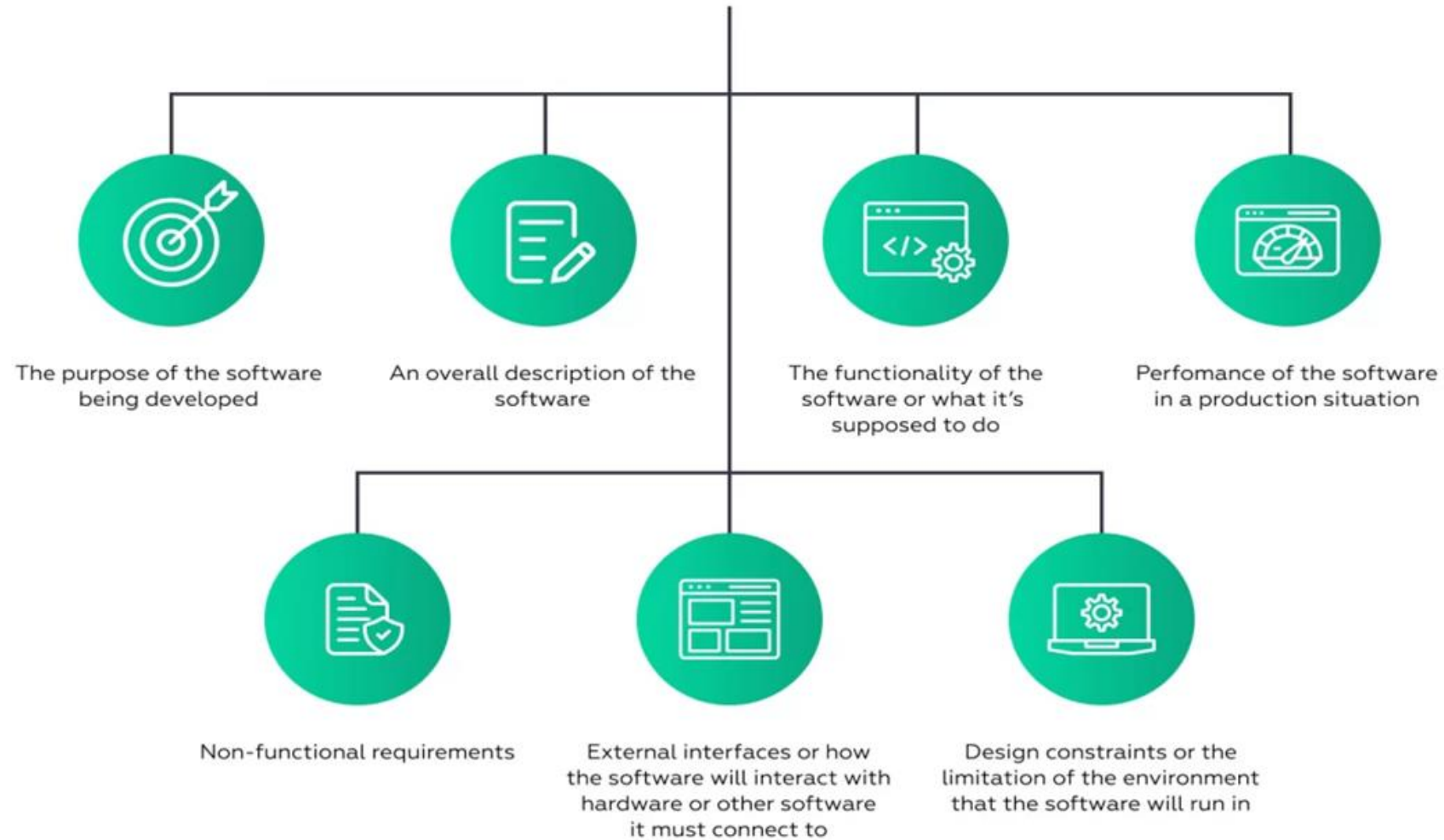
- Provide feedback to the customer, ensuring that the IT company understands the issues the software system should solve and how to address those issues.
- Help to break a problem down into smaller components just by writing down the requirements.
- Speed up the testing and validation processes.
- Facilitate reviews.

3 Reasons Why an SRS Document is Important

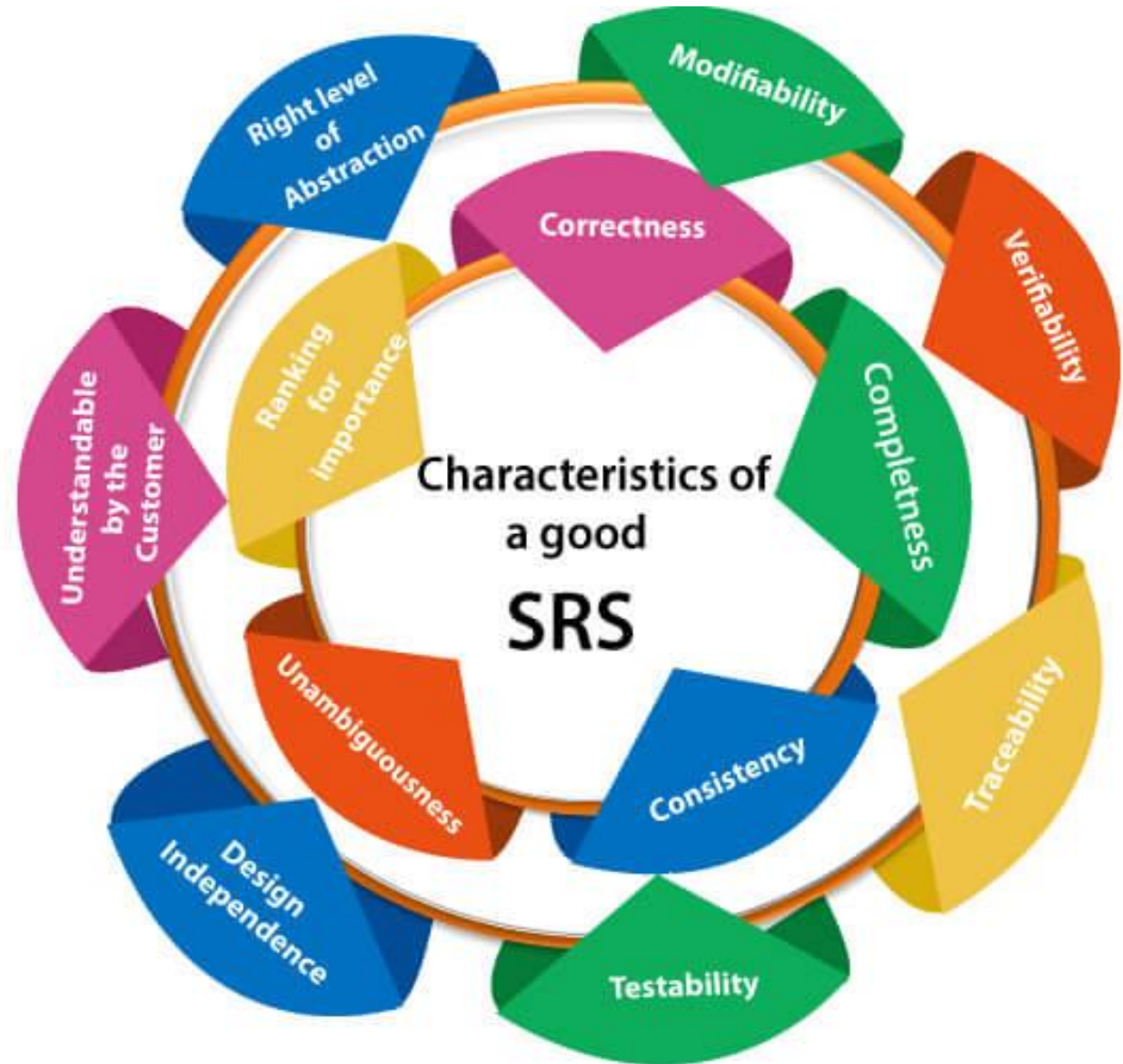
- SRS is a Communication Point
- SRS Includes Final Description of Features
- SRS is a Standard Guide for Product Developers



SRS Document Elements



Characteristics of good SRS



Characteristics of good SRS (cont'd)

- **Correctness:**

User review is used to ensure the correctness of requirements stated in the SRS. SRS is said to be correct if it covers all the requirements that are actually expected from the system.

- **Completeness:**

Completeness of SRS indicates every sense of completion including the numbering of all the pages, resolving the to be determined parts to as much extent as possible as well as covering all the functional and non-functional requirements properly.

- **Consistency:**

Requirements in SRS are said to be consistent if there are no conflicts between any set of requirements. Examples of conflict include differences in terminologies used at separate places, logical conflicts like time period of report generation, etc.

Characteristics of good SRS (cont'd)

- **Unambiguousness:**

SRS is said to be unambiguous if all the requirements stated have only 1 interpretation. Some of the ways to prevent unambiguousness include the use of modelling techniques like ER diagrams, proper reviews and buddy checks, etc.

- **Ranking for importance and stability:**

There should a criterion to classify the requirements as less or more important or more specifically as desirable or essential. An identifier mark can be used with every requirement to indicate its rank or stability.

Characteristics of good SRS (cont'd)

- **Modifiability:**

SRS should be made as modifiable as possible and should be capable of easily accepting changes to the system to some extent. Modifications should be properly indexed and cross-referenced.

- **Verifiability:**

SRS is verifiable if there exists a specific technique to quantifiably measure the extent to which every requirement is met by the system. For example, a requirement stating that the system must be user-friendly is not verifiable and listing such requirements should be avoided.

- **Traceability:**

One should be able to trace a requirement to design component and then to code segment in the program. Similarly, one should be able to trace a requirement to the corresponding test cases.

Characteristics of good SRS (cont'd)

- **Design Independence:**

There should be an option to choose from multiple design alternatives for the final system. More specifically, the SRS should not include any implementation details.

- **Testability:**

SRS should be written in such a way that it is easy to generate test cases and test plans from the document.

- **Understandable by the customer:**

An end user maybe an expert in his/her specific domain but might not be an expert in computer science. Hence, the use of formal notations and symbols should be avoided to as much extent as possible. The language should be kept easy and clear.

- **Right level of abstraction:**

If the SRS is written for the requirements phase, the details should be explained explicitly. Whereas, for a feasibility study, fewer details can be used. Hence, the level of abstraction varies according to the purpose of the SRS.



SRS document

- Once you have gathered all requirements of a project you are ready to begin writing the SRS document that describes what the software will do and how it will be expected to perform. It also describes the functionality the product needs to fulfil all stakeholders (business, users) needs.
- The two main participants in the SRS creation process are the contractor and the customer. The customer usually takes the most responsibility for the documentation. Sometimes, a customer has no idea exactly how the project must look. In other cases, he has too many ideas. In either case, cooperation with the contractor will help you to understand exactly what is needed and create the proper documentation.

Steps to write good SRS document

Step 1: Build an Outline

The software requirements specification must have a clear and understandable structure. A good outline is crucial to the whole software development process. It helps avoid wasting developers' time and effort reading unnecessary information. Furthermore, it minimizes the risk of excluding valuable data from the process.

Step 2: Determine the Project Goal

identify your project's purposes using the following process:

- 1- Clarify your product's scope.
- 2- Illustrate its value to consumers.
- 3- Specify the target audience for the product.
- 4- Showcase all the product's advantages.

Steps to write good SRS document

Step 3: Write an Overview of Your Future Project

In this stage, you will need to give an in-depth description of the project you're ultimately going to create. Mention all functions and features of the software and define how they will fit users' needs and expectations.

Step 4: Detail Your Functional and Non-Functional Requirements

After writing the general description of your software, it is time to define its specific requirements. Outlining the various types of requirements in software engineering functional requirements and non-functional requirement.

Step 5: Get Stakeholders Approval for the SRS Document

approval is a much-anticipated part of development. However, client's edits can sometimes negate the whole amount of work, completed at a certain point in the project. To avoid the pressure, we recommend agreeing to small parts of completed SRS documentation, rather than submitting a whole package of at once. This helps to fix certain issues quickly and is less troublesome.

What should a Typical SRS Document Include

Introduction

The structure of the SRS document begins with an introduction. This section illustrates the overall software requirements along with their scope for your team.

Purpose

It includes the types of requirements described in the document and a list of the people who will work with it. A few paragraphs will be enough to discuss the goals of your SRS documentation.

Intended Audience

List all the team members and stakeholders who will take part in the SRS document creation and those who will have access to the completed document. Describe their roles to identify each one's responsibilities within the project.

What should a Typical SRS Document Include (cont'd)

Intended Use

Identify the cases in which your team will use the software requirements specification:

- determining new functions through brainstorming
- estimating costs and project duration
- possible risks evaluation
- monitoring the team's success within the project
- dealing with conflicting situations

Scope

This subsection provides a brief representation of the system. You can describe the project from a more technical angle, mentioning its functionality and positioning.

Illustrate the users that are expected to interact with the system. Explain why this software development solution is unique. Thus, you will be able to interest stakeholders and gain more investments in your project.

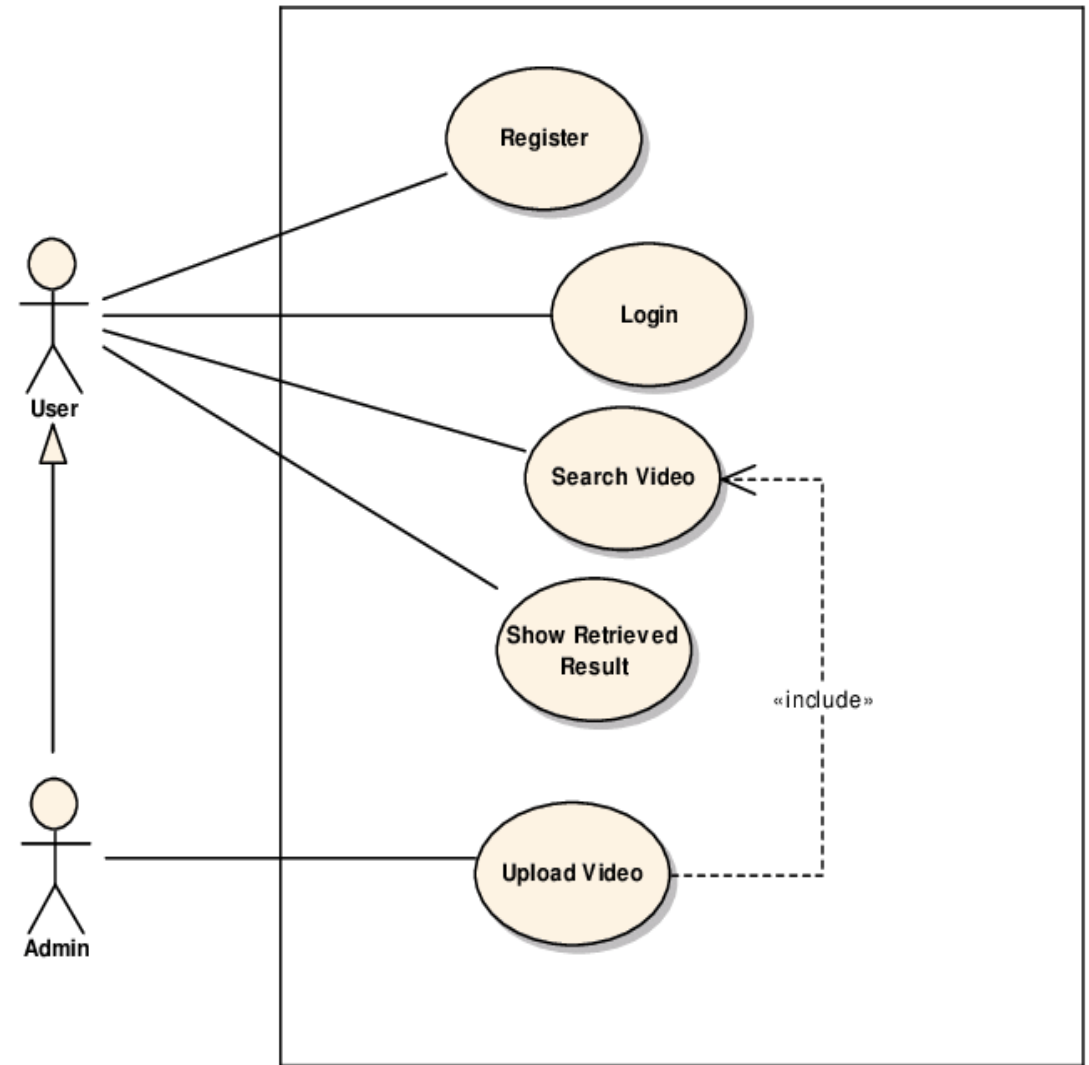
Eng. Asmaa Lafi

Definitions and Acronyms

Describe all specific terms mentioned in the requirements documentation in software engineering, to avoid misunderstandings with new developers. Name business entities and stakeholders with a particular abbreviation to speed up the process of SRS creation.

Tools for SRS Documentation

- **Use Case Diagram :** represent the dynamic behaviour of a system. It encapsulates the system's functionality by incorporating use cases, actors, and their relationships. It models the tasks, services, and functions required by a system/subsystem of an application. It depicts the high-level functionality of a system and also tells how the user handles a system.



Video search systems

Tools for SRS Documentation (cont'd)

User story: is a **requirement expressed from the perspective of an end-user goal**. User Stories may also be referred to as **Epics**, Themes or features but all follow the same format.

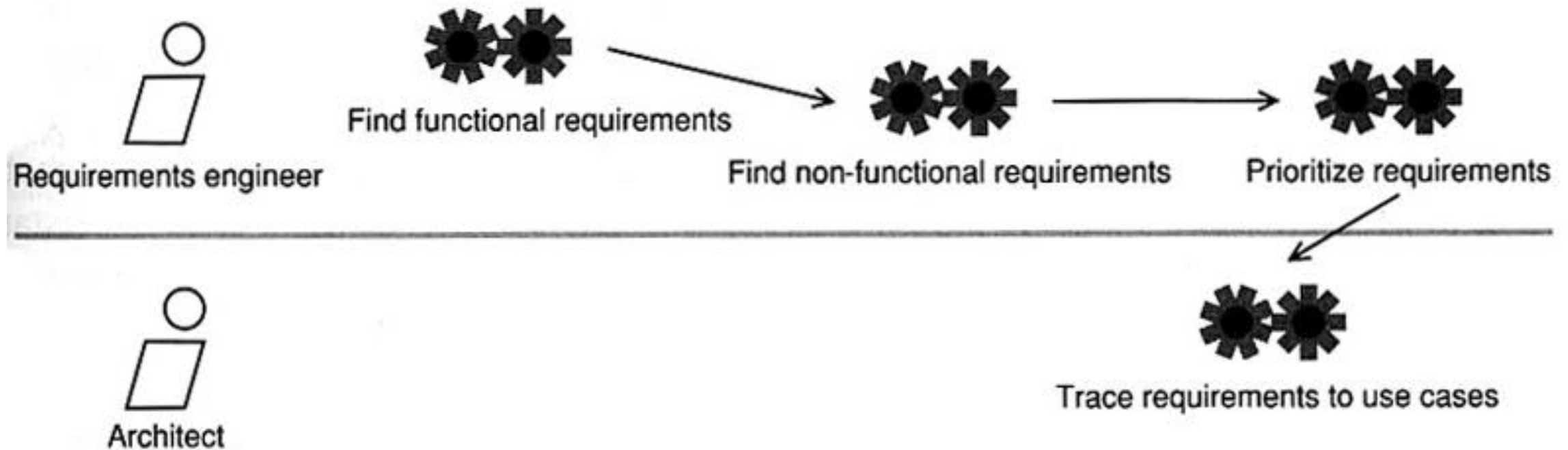
- A User Story is really just a well-expressed requirement.
- The User Story format has become the most popular way of expressing requirements in Agile for a number of reasons:
 - It focuses on the viewpoint of a role who will use or be impacted by the solution
 - It defines the requirement in language that has meaning for that role
 - It helps to clarify the true reason for the requirement
 - It helps to define high level requirements without necessarily going into low level detail too early
- User Stories are often deemed to comprise three elements - **the 3C's**
 - Card
 - Conversation
 - Confirmation

User Stories

Title:	Priority:	Estimate:
User Story: As a [description of user], I want [functionality] so that [benefit]		
Acceptance Criteria: Given [how things begin], I want [functionality] so that [benefit]		

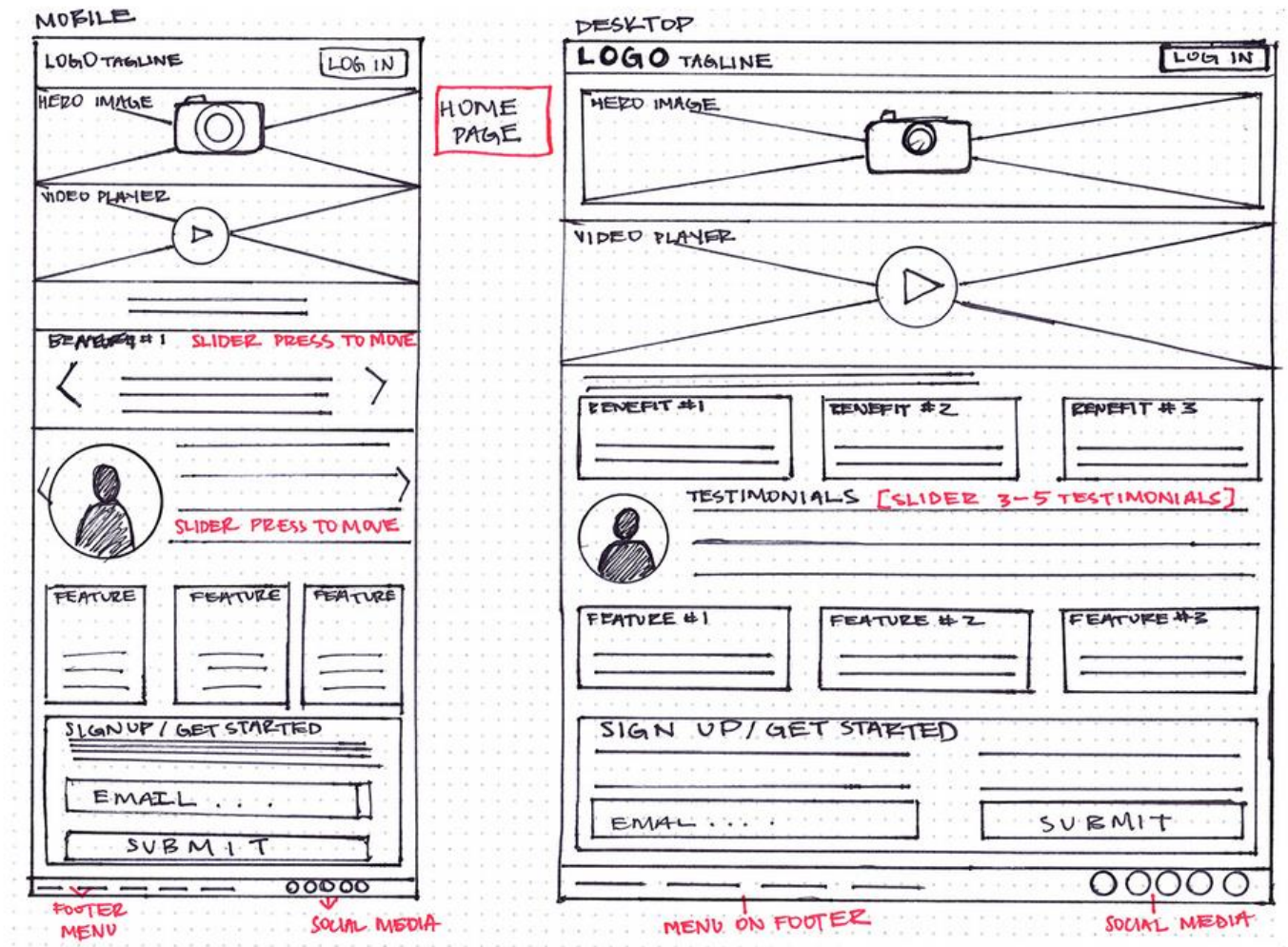
Tools for SRS Documentation (cont'd)

- **Workflow:** series of activities shows us the workers and activities involved in a particular workflow. A workflow specifies the component tasks and related roles of a specific workflow.



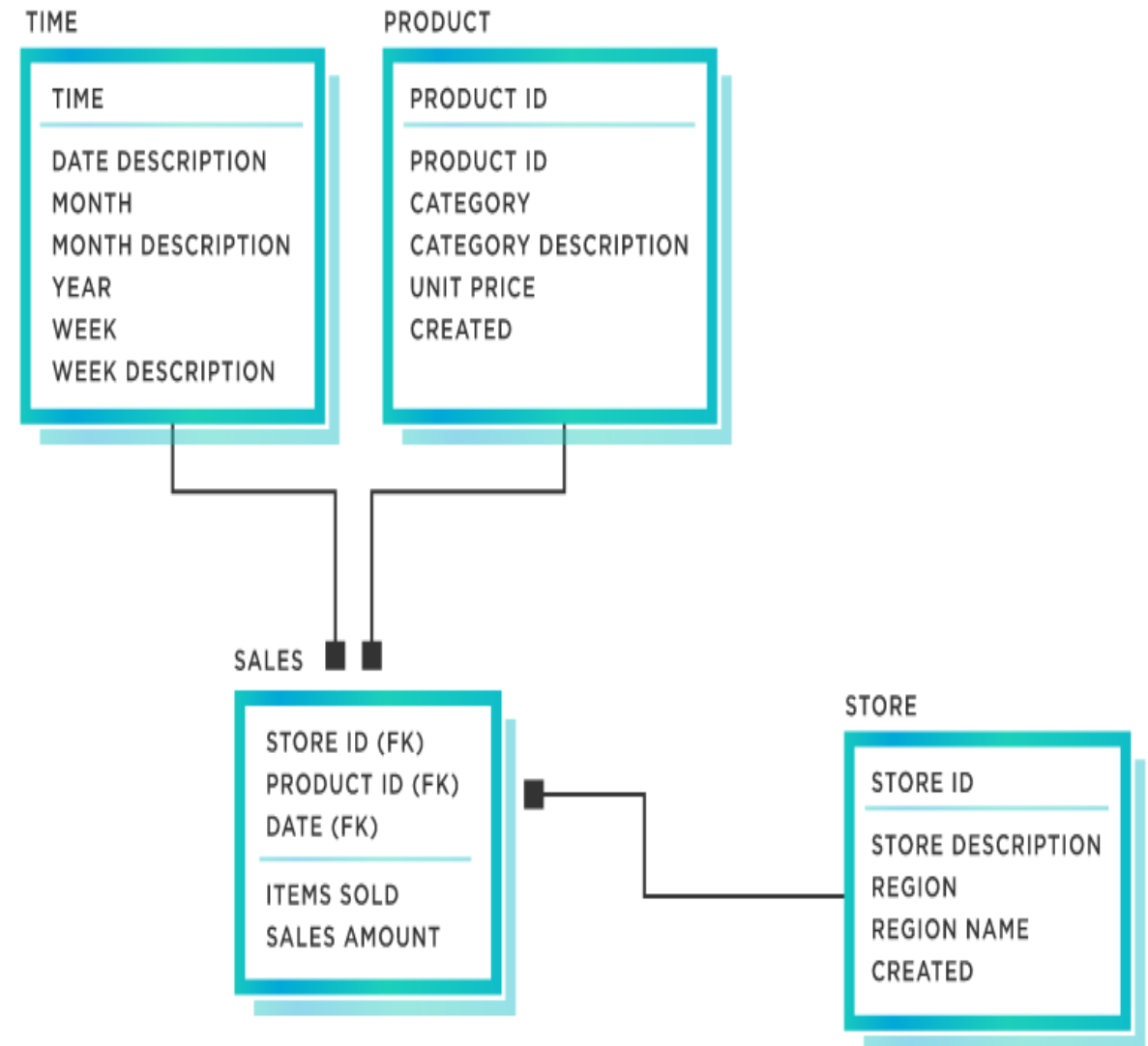
Tools for SRS Documentation (cont'd)

- **Wireframe:** tends to be more sketch-like and not have as much visual detail. Their job is to help visually describe functionality without getting bogged down on stylistic choices. **it helps understanding of the project requirements and thus eliminate confusions and vagueness early on**



Tools for SRS Documentation (cont'd)

- **Logical Data Model:** It serves to define how a system has to be implemented regardless of the database management system being used. **Data architects and business analysts** are usually the creators of a logical data model. The goal of creating a logical data model is to develop a highly technical map of underlying rules and data structures.



Tools for SRS Documentation (cont'd)

Data dictionary: is a file or a set of files that contain information about objects in the database. It's basically a repository of data names, definitions, and attributes used to describe the data. We can say its data about data.

DATA				
employee_id	first_name	last_name	nin	dept_id
44	Simon	Martinez	HH 45 09 73 D	1
45	Thomas	Goldstein	SA 75 35 42 B	2
46	Eugene	Comelsen	NE 22 63 82	2
47	Andrew	Petculescu	XY 29 87 61 A	1
48	Ruth	Stadick	MA 12 89 36 A	15
49	Bary	Scardelis	AT 20 73 18	2
50	Sidney	Hunter	HW 12 94 21 C	6
51	Jeffrey	Evans	LX 13 26 39 B	6
52	Doris	Bemdt	YA 49 88 11 A	3
53	Diane	Eaton	BE 08 74 68 A	1

DATA DICTIONARY (METADATA)		
Column	Data Type	Description
employee_id	int	Primary key of a table
first_name	nvarchar(50)	Employee first name
last_name	nvarchar(50)	Employee last name
nin	nvarchar(15)	National Identification Number
position	nvarchar(50)	Current postion title, e.g. Secretary
dept_id	int	Employee depamtnet. Ref: Departmetns
gender	char(1)	M = Male, F = Female, Null = unknown
employment_start_date	date	Start date of employment in organization.
employment_end_date	date	Employment end date.

References

- Requirements Engineering: From System Goals to UML Models to Software Specifications
- <https://docode.dev/software-requirement-specification-guide/>
- <https://www.techtarget.com/searchsoftwarequality/definition/software-requirements-specification>
- <https://www.techtarget.com/searchsoftwarequality/definition/software-requirements-specification>
- <https://www.javatpoint.com/software-requirement-specifications>
- <https://impressit.io/blog/software-requirements-specification-guide>
- <https://www.uptech.team/blog/srs-document>
- <https://www.tibco.com/reference-center/what-is-a-logical-data-model>